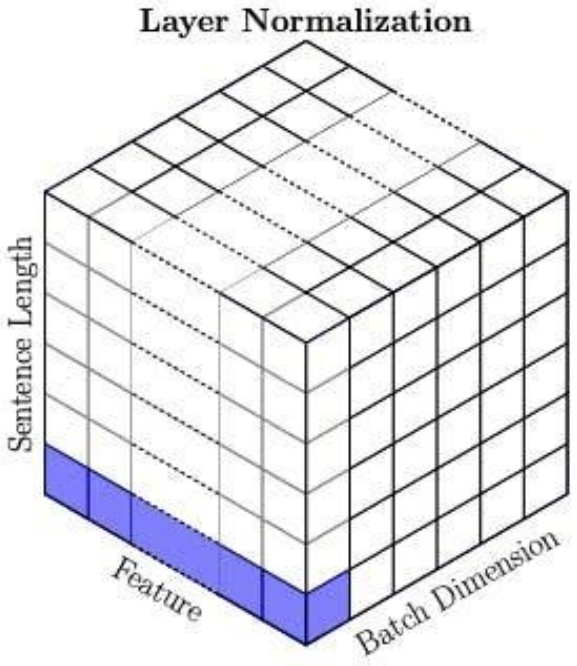
학습 성능 향상 방법

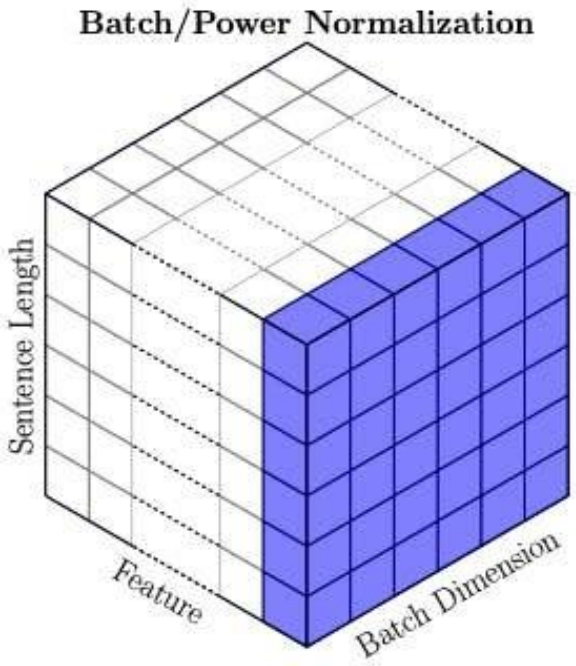
정규화, 가중치초기화, 정칙화, 드롭아웃, 그레이디언트 클래핑

1. 정규화

데이터의 범위를 일정한 구간으로 맞추는 것
데이터의 분산과 평균은 조작되지만 데이터의 상대적 관계는 보존
학습 데이터 자체를 정규화 할 수 있고, 모델 내부에 정규화 하는 계층을 넣을 수도 있음

Cat = [[1,2,3,4]]
Dog = [[5,6,7,8]]





1. 정규화

트랜스포머에서는 어떻게 구현될까? > Layernorm을 변형하여 사용함
Layernrom을 사용하는이유
(1) 배치 정규화는 미니배치에 의존적이며 배치마다 서로 다른 길이의 sequence 가 들어오게 됨
I am a student 4개 , I love you [PAD] 4개
(1) RNN과 같은 시계열 언어 모델의 경우 각 time-step마다 다른 통계값을 저장해야하기 때문에 모델의 복잡도가 증가함

1. 각 토큰의 값들에 대한 평균과 분산 계산
2. 평균0, 분산 1의 분포로 정규화
3. 학습 가능한 파라미터임 **감마(scale, γ)**와 **베타(shift, β)**를 곱하고 더해줌 ax + b

표준화를 마친 후 감마를 곱하고 베타를 더하는 Linear 계층을 추가하는 이유
데이터의 표현성을 증가시키기 위함: 단순 표준화에서는 "모든 입력값은 무조건 평균 0, 분산 1이어야 해"라는 제약 때문에 모델의 표현력(Representational Power)이 제약을 받음

결과적으로 각 특성값들이 평균 0이고 분산이 1인 근처 범위에서 놀게됨

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

2. 가중치 초기화

가중치 초기화의 가장 단순한 방식은 상수 초기화이다> 모든 노드가 동일한 출력을 생성하도록 학습하는 대칭 파괴(Breaking Symmetry) 현상이 발생한다.

무작위 초기화의 경우에도 노드의 출력값이 양 끝단으로 치우쳐져 기울기 소실 현상이 발생할 수 있다.

Xavier, He 초기화 : 신경망의 분산과 정규분포/균등분포를 기반으로 초기화하는 방식으로 노드를 출력한 통계값이 일정한 분산을 최대한 유지할 수 있도록 하는 초기화 방식이다.

파이토치에는 기본 가중치 초기화를 지원하지만 ReLU계열 활성화 함수를 사용할 때에는 허 초기화, 시그모이드를 사용할 때에는 제이비어 초기화를 지정해야한다.

각각의 노드를 통과하면 분산은 변화하지 않는지?
맞음 > 초기화의 목적은 각 레이어의 통과 전후에 활성화 값의 분산 스케일이 크게 달라지지 않도록 조절하는 것. > 학습의 안정성 도모

2. 가중치 초기화

```python
# 실습1
class MyModel1(nn.Module):
  def __init__(self):
    super().__init__()
    self.layer = nn.Sequential(
        nn.Linear(1,2),
        nn.Sigmoid()
    )
    self.fc = nn.Linear(2,1)
    self._init_weights()

  def forward(self, x):
    output = self.layer(x)
    output = self.fc(output)
    return output

  def _init_weights(self):
    nn.init.xavier_uniform_(self.layer[0].weight) # 가중치와 편향이 있는 Linear 계층만 초기화해야 한다.
    self.layer[0].bias.data.fill_(0.01) # 편향 값

    nn.init.xavier_uniform_(self.fc.weight)
    self.fc.bias.data.fill_(0.01) # 편향 값

model = MyModel1()
print(model)
```

2. 가중치 초기화

```
# 실습2
# apply 함수 사용 (모델의 모든 하위 모듈(submodule)에 사용자 정의 함수를 재귀적으로 적용할 때 사용)
class MyModel1(nn.Module):
  def __init__(self):
    super().__init__()
    self.layer = nn.Sequential(
        nn.Linear(1,2),
        nn.Sigmoid()
    )
    self.fc = nn.Linear(2,1)
    self.apply(self._init_weights)

  def forward(self, x):
    output = self.layer(x)
    output = self.fc(output)
    return output

  def _init_weights(self, module):
    if isinstance(module, nn.Linear):
      nn.init.xavier_uniform_(module.weight) # 가중치와 편향이 있는 Linear 계층만 초기화해야 한다.
      nn.init.constant_(module.bias, 0.01)
      print('가중치 초기화 적용 완료')

model = MyModel1()
print(model)
```

module.weight

3. 정칙화 (Regularization)

모델이 암기가 아니라 일반화를 할 수 있도록 규제를 가하는 것

노이즈를 임의로 추가하여 모델의 일반화 성능을 높인다.

L1 (lasso) 정칙화 : 가중치합에 절댓값을 적용하고 규제강도를 곱하여 손실함수에 더함 (0으로 수렴)
L2 (ridge) 정칙화 : 가중치에 평균제곱오차를 적용하여 규제강도를 곱하고 손실함수에 더함
정칙화는 모델의 과대적합을 억제한다는 장점이 있으나 손실계산 과정에서 모델의 복잡도가 증가한다 (매 과정에서 가중치 합을 계산해야 하기 때문) > 대부분 언어모델에서 사용

$$\text{Loss} = \text{Original Loss} + \lambda \sum_{i=1}^{n} |w_i|$$

$$\text{Loss} = \text{Original Loss} + \frac{1}{2} \lambda \sum_{i=1}^{n} w_i^2$$

4. 가중치 감쇠 (Weight Decay)

모델이 더 작은 가중치를 갖도록 규제를 가하는 것
파이토치의 가중치 감쇠는 L2 정칙화와 동의어이다.

Plain Text

```
optimizer = torch.optim.SGD(model.parameters(), lr = 0.01, weight_decay = 0.01)
```

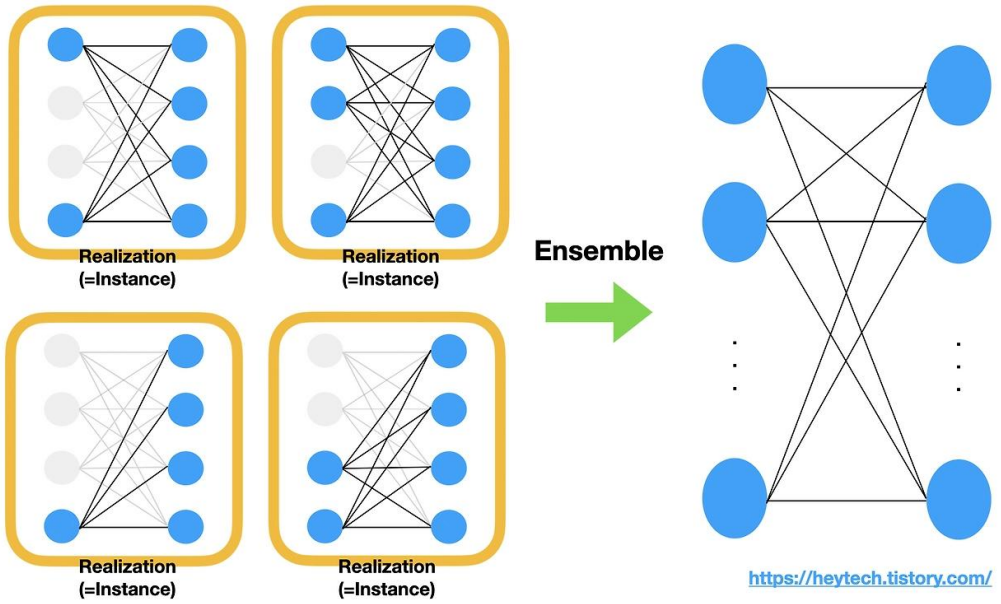
5. 드롭아웃 (Dropout)

모델 학습 과정 중 일정 비율의 노드를 없애거나 0으로 만들어버리는 것
동조화(Co-Adaptation) 문제: 모델 학습 중 특정 노드의 가중치나 편향이 큰 값을 갖게 되면 다른 노드가 큰 값을 갖는 노드에 의존하는 것

기존 기계학습에서 다뤄진 앙상블(Ensemble) 기법의 연장선이다.
model.eval()을 사용하는 이유

```
class MyModel1(nn.Module):
  def __init__(self):
    super().__init__()
    self.layer1 = nn.Linear(10, 10)
    self.dropout = nn.Dropout(p = 0.5) # layer1에서 50프로의 노드를 0으로 만들어버림
    self.layer 2 = nn.Linear(10, 10)

  def forward(self, x):
    x = self.layer1(x)
    x = self.dropout(x)
    x = self.layer2(x)
```



6. 그레이디언트 클리핑

가중치가 일정 기울기를 넘어서면 기울기를 잘라 설정한 기울기로 바꿔주는 것
RNN 과 LSTM 학습에 주로 사용된다
Hyperparameter tuning 에서 하이퍼파라미터가 결정

```
for x, y in train_dataloader:
    x = x.to(device)
    y = y.to(device)
    output = model(x)

    loss = criterion(output, y)
    optimizer.zero_grad()
    loss.backward()

    torch.nn.utils.clip_grad_norm(model.parameters(), 0.1) # max_norm = 1.0

    optimizer.step()

    cost += loss
```

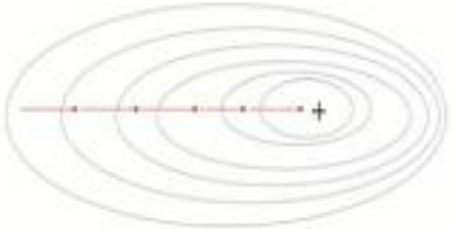
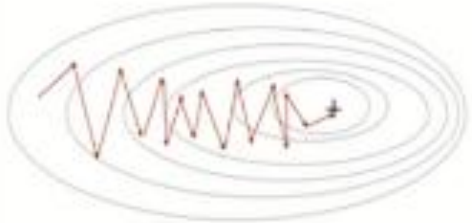
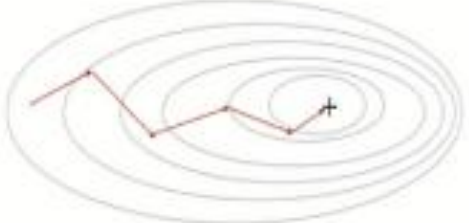
최적화 알고리즘 (Optimizer)

GD > SGD, BGD > Momentum > AdaGrad, RMSProp > Adam, AdamW

발전과정 1. 배치 개념

GD : 단순 경사하강법

SGD : 확률적 경사하강법 (무작위), 배치, 미니배치 경사하강법

	경사하강법	확률적 경사하강법	미니배치 경사하강법
해를 찾는 과정	Gradient Descent 	Stochastic Gradient Descent 	Mini-Batch Gradient Descent 
1 Iteration에 사용되는 데이터 크기	모든 데이터	임의의 하나의 데이터	설정한 미니배치 사이즈만큼

발전과정 2. 모멘텀

Gradient Descent with Momentum

$v_{dw} = \beta * v_{dw} + (1-\beta) * dw$ | 이때, $dw = \frac{d}{dw} * J(w,b)$

$v_{db} = \beta * v_{db} + (1-\beta) * db$ | 이때, $db = \frac{d}{db} * J(w,b)$

$w = w - \alpha * v_{dw}$

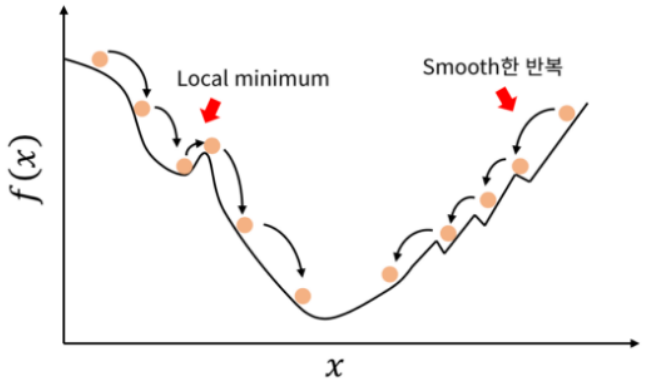
$b = b - \alpha * v_{db}$

이전 시간의 기울기까지 고려하여 가중치를 업데이트

속도(Velocity) 개념을 도입한다.

관성 Momentum

돌이 굴러 떨어지듯, 이동 벡터를 이용해 이전 기울기에 영향을 받도록 하는 방법



$$v_t = \gamma v_{t-1} + \eta \nabla f(x_{t-1})$$
$$x_t = x_{t-1} - v_t$$

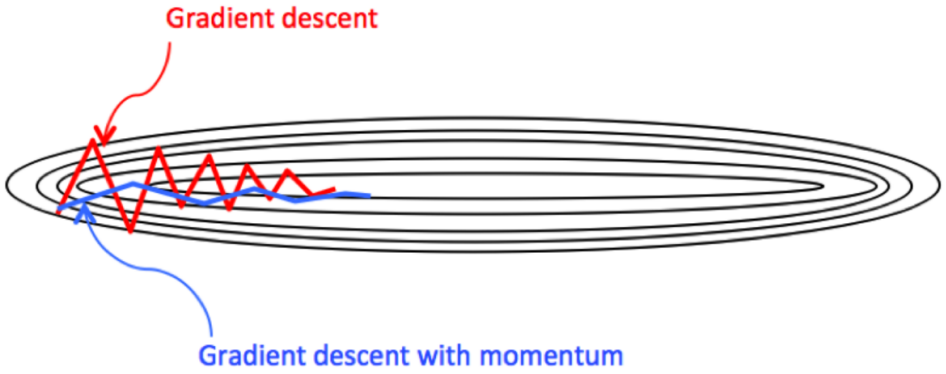
γ : 관성 계수 (momentum term) ≈ 0.9

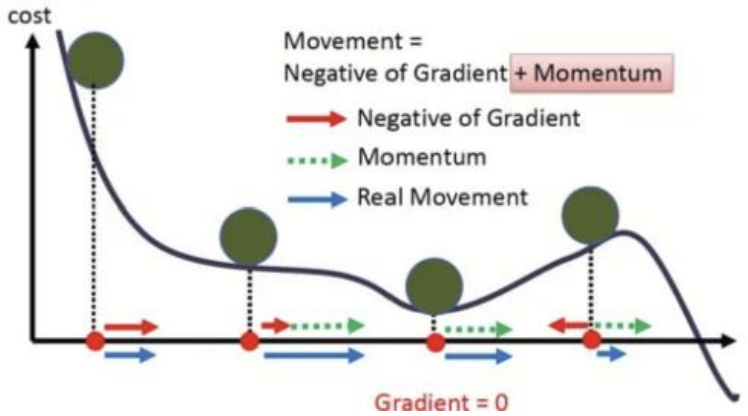
η : 학습율 (Learning rate)

v_t : t번째 step에서의 x의 이동 벡터

관성(Momentum)을 이용하면 Local minimum과 잡음에 대처할 수 있다.

이동 벡터(v_t)를 추가로 사용하므로, 경사 하강법 대비 2배의 메모리를 사용한다.





발전과정 3. 적응형

Adagrad : 학습률을 매개변수에 맞춰 자동으로 조절하자.
많이 이동한 학습률은 줄이기

RMSProp : Adagrad는 학습이 길어지면 학습 정체 발생.
지수이동평균 개념을 도입해서 먼 과거의 기울기는 서서히 잊고 최근의 기울기에 더 높은 가중치 반영

발전과정 5. 융합

Adam : 모멘텀 + 적응형 알고리즘
AdamW : 현재의 표준 알고리즘
- 기존 Adam 알고리즘은 L2규제와 충돌함
- 기울기가 큰 가중치는 패널티를 적게 받아버림

수식을 수정하여 기울기 이동 후 가중치 크기를 줄이도록 수정

- 1차 모멘텀: 가던 방향으로 가라
- 2차 모멘텀 : 보폭 제어
- 편향 보정 : 학습 초기 모멘텀 0 제어

1. **1차 모멘텀(Momentum, 평균) 계산**

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

2. **2차 모멘텀(Uncentered Variance, 제곱 평균) 계산**

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

3. **편향 보정(Bias Correction)**

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t}$$

4. **가중치 업데이트 (Weight Decay 분리 적용)**

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right)$$

1. η: 학습률 (Learning Rate)
2. λ: 가중치 감쇠(Weight Decay) 계수
3. ε: 0으로 나누는 것을 방지하기 위한 작은 수